Software Developer Coding Test

Background:

In this position, you will be working with our client who runs one of the fastest growing startups in Europe. We are on a mission to construct a scalable scraping pipeline, targeting legal texts from around the globe. As part of this ambitious challenge, we need your expertise to develop a scraper for one of Ireland's premier legal resources.

Solve the following tasks below and send your solution back by sharing a **git** repository containing the solution files. **Please don't forget to include a .MD file containing instructions to run the code.**

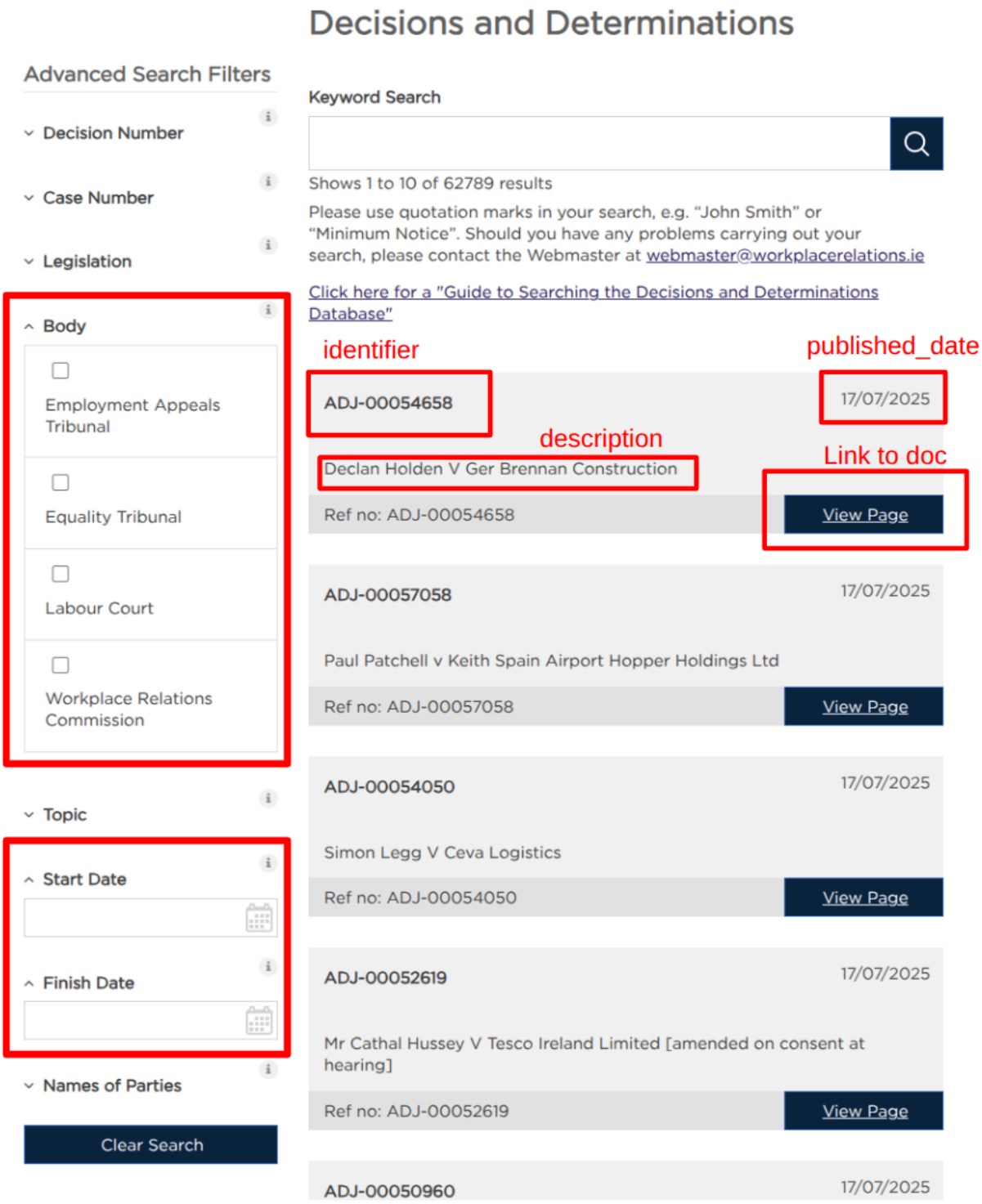
Your task is to create a pipeline using the Scrapy framework to scrape documents and metadata from the website Workplace Relations. This project includes downloading documents to local storage and extracting essential metadata.

You may use AI tools freely, but you must be able to explain every design decision, trade-off, and line of code in your solution during the technical interview. We evaluate understanding, not just output.

Requirements:

Scraping

Use the **Scrapy framework** to create the scraper



1. Make sure to use the fastest way to scrape the URLs without getting blocked.
2. Scrape from each of the bodies on the **left side**, and use the **start** and **end** date filters to partition the scraping process
3. The scraper must get **start_date** and **end_date** inputs to fetch documents in a specific time period, and the scraper must iterate on a time-period basis between the two dates. (example monthly partitions between 01-01-2024 and 01-01-2025), and add a **partition_date** field for every record
4. Extract **metadata** and capture the details for each record (title, description identifier, date, link to doc, partition_date, etc.)
5. Store the **metadata** in a NoSQL DB of your choice
6. Download and store the document files in a blob/object storage
    a. If the records have links to **PDF/DOCS** formatted documents, download and store them as they are.
    b. If the records have links to an **HTML** page, navigate to that page, scrape **all the web pages**, and store them as `.html` files.
7. Store the **path-to-the-file** in its respective metadata record in the NoSQL DB
8. Calculate the **file_hash** of the file and store it in its respective metadata record in the NoSQL DB
9. Your pipeline must be **idempotent**. Running it twice on the same date range must not create duplicate records or re-download unchanged files. Use the file hash to detect changes between runs.
10. Produce **structured logs** (JSON format) that include: the current partition being processed, the body being scraped, number of records found vs. successfully scraped, any failed downloads with URLs and error codes, and a summary at the end of each run.

Infrastructure

- **Storages**
    - Use a NoSQL DB and an object storage in **docker containers**
- **Orchestration**
    - Use one of the following to orchestrate the ingestion and transformation as **separate tasks with proper dependency handling**: Dagster, Airflow, or Modal. Running from the command line is acceptable but using an orchestrator will carry significant weight in the assessment.
- **Configuration**:
    - All connection strings, storage paths, partition sizes, and scraping parameters must be configurable via environment variables or a config file. No hardcoded values.

Transformation script

1. A Python script to run some transformations on the **Landing Zone** data
    a. Given a **start date** and an **end date**, it should fetch metadata from mongo
    b. Gets the relative files from the object/blob storage
    c. Iterates through them
        i. If the file is a **pdf/doc** file, don't apply any transformation
        ii. If the file is an **html** file
            1. Use an html parser like **BeautifulSoup**, and get **only the relevant content** of the document (excluding navigation bars/buttons, headers, footers, etc.) **Example in the screenshot below.**
            2. Calculate the new **file_hash** value of the file

iii. Change the name of ALL the files to become **identifier.ext** (from **metadata**)
iv. Store all the docs in a new object/blob storage container
v. Store the metadata in a new NoSQL collection (with the new file path and file hash)

English | Gaeilge AAA

WRC

The Labour Court

This website contains decisions and determinations of both the Labour Court and the WRC. It also contains decisions and recommendations of the Equality Tribunal since 1996 and post 2007 determinations of the Employment Appeals Tribunal. For more information click here

Return to Search

IR - SC - 00001595

ADJUDICATION OFFICER Recommendation on dispute under Industrial Relations Act 1969

Investigation Recommendation Reference: IR - SC - 00001595

Parties:

Relevant content

	Worker	Employer
Anonymised Parties	*Employee*	*Health Service*

Representatives	Mes. Deirdre Canty, SIPTU	Self-Represented

Dispute:

Act	Dispute Reference No.	Date of Receipt
Complaint seeking adjudication by the Workplace Relations Commission under section 13 of the Industrial Relations Act, 1969	IR - SC - 00001595	01/08/2023

Workplace Relations Commission Adjudication Officer: Brian Dolan

Date of Hearing: 05/04/2024

Architecture Write-up:

Include an ARCHITECTURE.md file (1 page max) covering: why you chose your date partition size, how you handle retries and rate limiting, your deduplication strategy, and what you'd change if this pipeline needed to support 50+ sources.

Tips:

- We will evaluate your solution against approximately 500–1000 documents. Ensure your pipeline handles this volume reliably, but design it as if it needed to handle 1000x that.
- Feel free to add additional steps in the **Transformation** logic for better data quality
- Don't delete/update any of the stored data in the **Landing Zone**
- Make sure that your code is well documented, readable, and scalable
- Ensure robust error and exception handling and logging
    - If the date range (e.g. 2024 till 2025) on the website contain 200 records, the scraper should scrape 200 records, or scrape 200-X, where X is the number of docs not scraped for any reason and every single record from X is logged with the reason.
- Make sure that you follow python best practices